

Experiment No: 01

Experiment Name: Introduction to the Ubuntu Operating System.

Objective:

1. To understand the fundamental concepts and architecture of the Ubuntu OS.
2. To learn the key features of Linux-based systems (Open Source, Security, Multi-user).
3. To explore the Ubuntu Graphical User Interface (GUI) and the Command-Line Interface (CLI).
4. To understand basic system operations and the role of the Kernel.

Theory:

Ubuntu is a complete Linux operating system, available with both community and professional support. It is based on the Debian architecture and follows a regular release cycle (LTS versions every two years). As a Linux-based system, it sits between the hardware and the user, managing resources through the Kernel.

Key characteristics include:

1. Open Source: The source code is free to study, change, and distribute.
2. File System: Unlike Windows, it uses a hierarchical tree structure starting with the root (/).
3. Security: It features strong permission controls (Read, Write, Execute) and a robust sudo (SuperUser DO) system to prevent unauthorized system changes.
4. Terminal: The power of Ubuntu lies in its terminal, which allows users to interact directly with the OS using a shell (usually Bash).

Procedure / Algorithm:

1. Installation/Booting: Boot the computer using an Ubuntu Live USB or launch it in a Virtual Machine.
2. User Interface: Explore the GNOME Desktop Environment, including the Activities overview, the Dock (Launcher), and System Settings.
3. Terminal Access: Open the terminal using the keyboard shortcut `Ctrl + Alt + T`.
4. System Identification: Type basic commands to identify the system version and current user identity.
5. Help System: Use the `man` (manual) command to understand how other commands work.

Code / Command:

Enter these basic commands to interact with the Ubuntu environment:

```
lsb_release -a          # Displays the specific Ubuntu version and
distribution info
uname -r                # Shows the current Linux Kernel version
whoami                  # Displays the name of the currently logged-in user
pwd                     # Shows the 'Print Working Directory' (where you are
currently)
ls                       # Lists the files and folders in the current
directory
```

Output / Result:

The system responds with specific hardware and software information:

```
Distributor ID: Ubuntu
Description:    Ubuntu 24.04 LTS
Release:       24.04
Codename:      noble

Kernel Version: 6.8.0-31-generic
Current User:   student_user
Current Path:   /home/student_user
```

Conclusion:

Through this experiment, we successfully explored the Ubuntu environment. We learned that Ubuntu provides a secure and stable platform that is highly customizable. By using the terminal, we realized that command-line interactions offer more precision and power over system tasks than the GUI alone. This foundational knowledge is essential for performing advanced OS tasks and programming.

Discussion:

During the experiment, we discussed the "Sudo" philosophy—why Ubuntu disables the root account by default and requires users to use `sudo` for administrative tasks to prevent accidental system damage. We also compared Ubuntu to other operating systems like Windows and macOS, noting that Linux offers much better transparency regarding system processes. Beginners might find the lack of "C:" or "D:" drives confusing, but once the unified tree structure (/) is understood, file management becomes more logical and centralized.

Experiment No: 02

Experiment Name: Study and Implementation of Basic Linux Commands (Part 1: Directory Operations)

Objective:

1. To master navigation within the Linux hierarchical file system.
2. To understand the creation, deletion, and management of directories.
3. To learn the difference between absolute and relative paths.

Theory:

The Linux file system is organized in a tree-like structure starting from the root directory (/). Unlike Windows, which uses drive letters (C:, D:), Linux treats everything as a file. Directory operations are fundamental for organizing data. The shell provides specific utilities to manipulate these directories. Understanding paths is crucial: an **Absolute Path** starts from the root (e.g., /home/user/docs), while a **Relative Path** starts from the current location (e.g., Documents/). Commands like `mkdir` allow for directory creation, while `cd` (change directory) and `pwd` (print working directory) are essential for movement. The `ls` command is used to view the contents and properties of these directories.

Procedure / Algorithm:

1. Open the Terminal using `Ctrl + Alt + T`.
2. Check the current location using `pwd`.
3. Create a new directory structure using `mkdir`.
4. Move into the newly created directory using `cd`.
5. Create sub-directories to understand nested structures.
6. Navigate back to the parent directory using `cd ..`.
7. Remove an empty directory using `rmdir`.

Code / Command:

```
pwd                                # Identify current path
mkdir -p Lab2/Subdir1             # Create nested directories
ls -F                             # List with file type indicators
cd Lab2                           # Change to Lab2
ls -R                             # Recursive listing
cd ..                             # Return to previous level
rmdir Lab2/Subdir1                # Remove sub-directory
```

Output / Result:

The system successfully displayed the working directory path. New directories were created and verified using the `ls` command. Navigation between parent and child directories was smooth, and the recursive listing showed the entire sub-tree structure correctly.

Conclusion:

In this experiment, we gained hands-on experience with directory management in Ubuntu. We successfully performed creation, navigation, and deletion tasks. These commands form the backbone of terminal-based workflow, allowing a user to manage large sets of data efficiently without relying on a graphical file manager.

Discussion:

We observed that Linux is case-sensitive; "Folder" and "folder" are treated as distinct entities. We also explored the `-p` flag in `mkdir`, which is extremely useful for creating parent and child directories simultaneously. A key takeaway was the importance of the `cd ..` and `cd ~` shortcuts for quick navigation. We noted that `rmdir` only works on empty directories, which is a safety feature to prevent accidental data loss.

Experiment No: 03

Experiment Name: Study and Implementation of Basic Linux Commands (Part 2: File Operations)

Objective:

1. To perform file creation, copying, moving, and renaming.
2. To understand file permissions and hidden files.
3. To learn how to view file content directly from the terminal.

Theory:

File operations are the core of any operating system. In Linux, files are handled using commands like `touch` (for creation), `cp` (copy), and `mv` (move/rename). To view content, commands like `cat`, `more`, and `less` are used. A unique feature of Linux is the use of a "dot" (.) prefix to hide files. Furthermore, every file has metadata including ownership and permissions. The `ls -l` command reveals these details, showing who can read, write, or execute a file.

Procedure / Algorithm:

1. Create a text file using `touch`.
2. Add text to the file using redirection `>`.
3. Read the content using `cat`.
4. Copy the file to a new name using `cp`.
5. Rename the copied file using `mv`.
6. Display all files including hidden ones using `ls -a`.
7. Delete the files using `rm`.

Code / Command:

```
touch my_file.txt           # Create empty file
echo "OS Lab Experiment" > my_file.txt # Add content
cat my_file.txt             # View content
cp my_file.txt copy.txt     # Copying
mv copy.txt renamed_file.txt # Renaming
ls -la                     # Long list including hidden files
rm my_file.txt             # Deleting a file
```

Output / Result:

The terminal correctly created the text file and displayed the string "OS Lab Experiment". The copy and rename operations were verified by listing the directory contents. Hidden files (like `.bashrc`) were visible when using the `-a` flag.

Conclusion:

We successfully implemented various file manipulation techniques. This experiment highlights the efficiency of the CLI in handling multiple files quickly. Understanding how to create, modify, and delete files via command line is a prerequisite for system administration and shell scripting.

Discussion:

During the experiment, we noted that the `>` operator overwrites file content, while `>>` appends to it. This distinction is critical for data integrity. We also discussed the power of the `rm` command; unlike Windows, Linux does not have a "Trash" for terminal deletions, meaning once a file is removed, it is difficult to recover. The use of flags like `-i` with `rm` was recommended to provide a confirmation prompt before deletion.

Experiment No: 04

Experiment Name: Study and Usage of VI Text Editor in Linux

Objective:

1. To gain a deep understanding of the modal interface of the VI editor.
2. To master the transitions between Command Mode, Insert Mode, and Last Line Mode.
3. To perform text manipulation tasks such as editing, deleting, searching, and saving without a graphical interface.

Theory:

The VI editor (short for Visual Editor) is the most widely used text editor in the Unix/Linux world. It is highly efficient because it is designed to be used entirely from the keyboard. Unlike modern editors, VI is modal, meaning it has different behaviors depending on the current mode:

1. **Command Mode:** This is the starting mode. Every key pressed is interpreted as a command (e.g., `x` deletes a character, `dd` deletes a line). You cannot type text in this mode.
2. **Insert Mode:** This mode allows you to enter text into the file. You enter this mode by pressing `i` (insert), `a` (append), or `o` (open new line).
3. **Last Line (Ex) Mode:** Activated by typing `:` in command mode. It allows for file-level operations like saving (`:w`), quitting (`:q`), and global search/replace.

Understanding these transitions is key to Linux system administration, as VI is often the only editor available on remote servers or during system recovery.

Procedure / Algorithm:

1. Initialize the editor by typing `vi` followed by the filename in the terminal.
2. Switch to **Insert Mode** using the `i` key to begin writing the document.
3. Type the required text.
4. Switch back to **Command Mode** using the `ESC` key to navigate or edit text.
5. Use movement keys (`h`, `j`, `k`, `l`) to move the cursor.
6. Enter **Last Line Mode** by typing `:` to save changes and exit the editor.

Code / Command :

Follow these commands to simulate a real editing session:

```
# 1. Create and open a new file named 'lab_test.txt'
vi lab_test.txt

# 2. Press 'i' to enter INSERT MODE and type:
This is the Operating Systems Lab.
We are learning the VI editor.
Linux is powerful.

# 3. Press 'Esc' to return to COMMAND MODE.
# Practice these commands:
# 'yy' - Copy current line
# 'p'   - Paste line below
# 'dd' - Delete current line
# 'u'   - Undo last action

# 4. Press ':' to enter LAST LINE MODE and type:
:wq      # This stands for Write and Quit
```

Output / Result:

After exiting, use the `cat` command to verify the file content:

```
user@ubuntu:~$ cat lab_test.txt
This is the Operating Systems Lab.
We are learning the VI editor.
Linux is powerful.
```

The file was successfully created, edited, and saved using the modal interface of VI.

Conclusion:

In this experiment, we successfully explored the functionality of the VI editor. We learned how to toggle between different modes and perform basic text editing tasks. While the modal nature of VI requires practice, it provides a very fast and resource-efficient way to handle text files directly from the command line.

Discussion:

During the lab, we discussed the efficiency and learning curve of VI editor. Although it is harder to learn than GUI editors, it allows faster editing once mastered. We also learned about VIM, the improved version of VI with extra features. Finally, we noted the importance of commands like `:q!` to exit without saving changes.

Experiment No: 05

Experiment Name: Introduction to Shell Programming and Execution of Basic Scripts.

Objective:

1. To understand the concept of Shell and Shell Scripting.
2. To learn the basic syntax of Bash scripting (variables, input/output).
3. To write and execute a script that performs basic arithmetic operations.

Theory:

A Shell is a command-line interpreter that acts as an interface between the user and the Linux kernel. A Shell Script is a plain text file containing a sequence of commands that the shell executes as a program.

The first line of a script, `#!/bin/bash` (known as the Shebang), tells the system which interpreter to use. Shell scripting is used for automating repetitive tasks, system administration, and executing complex command sequences easily. Variables in shell scripts do not require data type declarations; they are treated as strings but can be used for arithmetic using the `(( ))` syntax.

Procedure / Algorithm:

1. Open the terminal (Ctrl+Alt+T).
2. Create a new file with a `.sh` extension: `vi basic_calc.sh`.
3. Add the Shebang line `#!/bin/bash` at the very top.
4. Use the `echo` command to print messages and the `read` command to take user input.
5. Perform arithmetic calculations using the `(( ))` expression.
6. Save and exit VI editor (`:wq`).
7. Provide execution permission to the file: `chmod +x basic_calc.sh`.
8. Run the script: `./basic_calc.sh`.

Code / Command:

Below is a script that takes two numbers from the user and calculates their sum and product.

```
#!/bin/bash

# Taking user input
echo "--- Basic Shell Calculator ---"
echo "Enter first number:"
read num1
echo "Enter second number:"
read num2
```

```
# Performing arithmetic operations
sum=$((num1 + num2))
product=$((num1 * num2))

# Displaying the results
echo "-----"
echo "The Sum of $num1 and $num2 is: $sum"
echo "The Product of $num1 and $num2 is: $product"
```

Output / Result:

```
--- Basic Shell Calculator ---
Enter first number:
12
Enter second number:
5
-----
The Sum of 12 and 5 is: 17
The Product of 12 and 5 is: 60
```

Conclusion:

In this experiment, we successfully created and executed our first Bash script. We learned how to handle user input, store data in variables, and perform basic mathematical operations. This confirms that shell scripts are a powerful way to combine individual Linux commands into a functional program.

Discussion:

During the execution, we found that the script will fail if the user enters text instead of numbers, as Bash is not strongly typed but expects integers for arithmetic. We also discussed the importance of the `chmod +x` command; without it, the script is just a text file and cannot be executed as a program. Using the `$` symbol before a variable name (like `$num1`) is essential to retrieve its stored value.

Experiment No: 06

Experiment Name: Implementation of Looping Constructs in Shell Programming.

Objective:

- To understand the logic and syntax of `for` and `while` loops in Bash.
- To learn how to automate repetitive tasks using iteration.
- To implement a script that generates a mathematical sequence (e.g., a multiplication table).

Theory:

Looping is a fundamental programming concept that allows a set of instructions to be executed repeatedly as long as a condition is met. In Shell scripting, there are three primary types of loops:

1. For Loop: Ideally used when the number of iterations is known. It can iterate over a range of numbers, elements in an array, or files in a directory.
2. While Loop: Executes a block of code as long as a specified condition evaluates to true. It is useful when the exact number of iterations depends on dynamic input.
3. Until Loop: Executes until a condition becomes true (opposite of while).

Loops are essential for system tasks like batch renaming files, monitoring system processes, or performing repetitive calculations without manual input.

Procedure / Algorithm:

1. Open the terminal and create a script: `vi multiplication.sh`.
2. Start with the Shebang line: `#!/bin/bash`.
3. Prompt the user to enter a number for which they want the multiplication table.
4. Use a For Loop starting from `i=1` to `i=10`.
5. Inside the loop, calculate the result: `n \times i`.
6. Print the result in a formatted manner.
7. Save, exit, and provide execution permissions using `chmod +x multiplication.sh`.
8. Run the script using `./multiplication.sh`.

Code / Command:

```
#!/bin/bash

# Script to generate a multiplication table
echo "--- Multiplication Table Generator ---"
echo -n "Enter a number: "
read num
```

```
echo "Multiplication Table for $num:"

# Using a C-style for loop for iteration
for (( i=1; i<=10; i++ ))
do
    result=$(( num * i ))
    echo "$num x $i = $result"
done
```

Output / Result:

```
--- Multiplication Table Generator ---
Enter a number: 7
Multiplication Table for 7:

7 x 1 = 7
7 x 2 = 14
7 x 3 = 21
7 x 4 = 28
7 x 5 = 35
7 x 6 = 42
7 x 7 = 49
7 x 8 = 56
7 x 9 = 63
7 x 10 = 70
```

Conclusion:

By completing this experiment, we successfully implemented iterative logic in a Linux environment. We learned how to control the flow of a program using the `for` loop construct and how to perform arithmetic within that loop. This demonstrates that Shell scripting is a viable tool for automating complex logical tasks.

Discussion:

During the lab, we discussed the various ways to write a `for` loop in Bash. While we used the C-style syntax `(( i=1; i<=10; i++ ))`, another common way is `for i in {1..10}`. We noted that the double parentheses `(( ))` are specifically used for integer arithmetic. We also discussed how loops can be combined with conditional statements (`if-else`) to create highly sophisticated automation tools, such as scripts that check for the existence of files before processing them.

Experiment No: 07

Experiment Name: Implementation of First Come First Serve (FCFS) CPU Scheduling Algorithm.

Objective:

1. To understand the working mechanism of the FCFS scheduling policy.
2. To learn how to calculate CPU scheduling metrics like Waiting Time (WT) and Turnaround Time (TAT).
3. To implement the algorithm using C programming and analyze its performance.

Theory:

First Come First Serve (FCFS) is the simplest CPU scheduling algorithm. It follows the FIFO (First-In-First-Out) policy, meaning the process that requests the CPU first is allocated the CPU first.

- Arrival Time (AT): The time at which a process enters the ready queue.
- Burst Time (BT): The time required by a process for its execution on the CPU.
- Waiting Time (WT): The total time a process spends waiting in the ready queue. $WT = \text{Turnaround Time} - \text{Burst Time}$.
- Turnaround Time (TAT): The total time taken from arrival to completion. $TAT = \text{Completion Time} - \text{Arrival Time}$.

A major drawback of FCFS is the Convoy Effect, which occurs when a process with a very long burst time occupies the CPU, causing all smaller processes behind it to wait for a long duration, leading to poor CPU utilization and high average waiting time.

Procedure / Algorithm:

1. Initialize an array to store Burst Times (BT) for n processes.
2. The Waiting Time (WT) for the first process is always 0 (assuming it arrives at time 0).
3. Calculate the WT for subsequent processes: $WT[i] = WT[i-1] + BT[i-1]$.
4. Calculate the Turnaround Time (TAT) for each process: $TAT[i] = WT[i] + BT[i]$.
5. Calculate the Average WT and Average TAT by summing all values and dividing by n.
6. Display the results in a tabular format showing Process ID, BT, WT, and TAT.

Code / Command:

```
cpp

#include <iostream>
using namespace std;

int main() {
    int n;

    cout << "Enter number of processes: ";
    cin >> n;

    int at[n], bt[n], ct[n], wt[n], tat[n];

    // Input Arrival Time & Burst Time
    for(int i = 0; i < n; i++) {
        cout << "\nProcess " << i+1 << " Arrival Time: ";
        cin >> at[i];
        cout << "Process " << i+1 << " Burst Time: ";
        cin >> bt[i];
    }

    // Sort by Arrival Time (FCFS rule)
    for(int i = 0; i < n-1; i++) {
        for(int j = i+1; j < n; j++) {
            if(at[i] > at[j]) {
                swap(at[i], at[j]);
                swap(bt[i], bt[j]);
            }
        }
    }

    // Calculate Completion Time
    ct[0] = at[0] + bt[0];

    for(int i = 1; i < n; i++) {
        if(ct[i-1] < at[i])
            ct[i] = at[i] + bt[i];
        else
            ct[i] = ct[i-1] + bt[i];
    }

    // Calculate TAT and WT
    for(int i = 0; i < n; i++) {
        tat[i] = ct[i] - at[i];
        wt[i] = tat[i] - bt[i];
    }

    // Output Table
    cout << "\nP\tAT\tBT\tCT\tTAT\tWT\n";
```

```

for(int i = 0; i < n; i++) {
    cout << "P" << i+1 << "\t" << at[i] << "\t" << bt[i]
        << "\t" << ct[i] << "\t" << tat[i]
        << "\t" << wt[i] << endl;
}

// Average Calculation
float total_wt = 0, total_tat = 0;

for(int i = 0; i < n; i++) {
    total_wt += wt[i];
    total_tat += tat[i];
}

cout << "\nAverage Waiting Time = " << total_wt / n;
cout << "\nAverage Turnaround Time = " << total_tat / n;

return 0;
}

```

Output / Result:

```

Enter total number of processes: 3
Enter Burst Time for each process:
P[1]: 24
P[2]: 3
P[3]: 3

Process      Burst Time      Waiting Time      Turnaround Time
P[1]          24              0                24
P[2]          3              24              27
P[3]          3              27              30

Average Waiting Time: 17.00
Average Turnaround Time: 27.00

```

Conclusion:

In this experiment, we successfully implemented the FCFS scheduling algorithm. We observed how the execution order is determined strictly by the arrival sequence. While the logic is straightforward and easy to implement, it is clear that FCFS is not optimal for systems where process lengths vary significantly.

Discussion:

We discussed the performance of FCFS in different scenarios. For instance, if the processes in the example output arrived in the order P2, P3, P1, the average waiting time would drop drastically from 17.0 to $(0+3+6)/3 = 3.0$. This highlights that FCFS performance is highly dependent on the order of arrival. We also noted that FCFS is non-preemptive, meaning once a process starts, it cannot be interrupted until it finishes its burst.

Experiment No: 08

Experiment Name: Implementation of Shortest Job First (SJF) CPU Scheduling Algorithm.

Objective:

1. To implement the Non-Preemptive Shortest Job First (SJF) scheduling algorithm.
2. To understand how sorting by burst time minimizes the average waiting time.
3. To compare the efficiency of SJF over FCFS.

Theory:

Shortest Job First (SJF) is a scheduling policy that selects the waiting process with the smallest execution (burst) time to execute next. It is considered optimal because it results in the minimum average waiting time for a given set of processes.

- Non-Preemptive: Once the CPU is allocated to a process, it holds it until it completes its burst.
- Starvation: A potential disadvantage where long processes wait indefinitely if short processes keep arriving in the system.

Procedure / Algorithm:

1. Initialize arrays for Process ID, Burst Time (BT), Waiting Time (WT), and Turnaround Time (TAT).
2. Input the number of processes and their respective Burst Times.
3. Sort the processes in ascending order based on their Burst Time using a sorting algorithm (like Selection or Bubble Sort).
4. Calculate WT for the first sorted process as 0.
5. For all subsequent processes: $WT[i] = WT[i-1] + BT[i-1]$.
6. Calculate Turnaround Time: $TAT[i] = WT[i] + BT[i]$.
7. Display the final table and the calculated averages.

Code / Command:

```
C++

#include <iostream>
#include <algorithm>
using namespace std;

int main() {
    int n;
    int bt[20], p[20], wt[20], tat[20];
    float avg_wt = 0, avg_tat = 0;

    cout << "Enter total number of processes: ";
    cin >> n;

    cout << "Enter Burst Time for each process:\n";
    for (int i = 0; i < n; i++) {
        p[i] = i + 1; // Store Process IDs
        cout << "P[" << p[i] << "]: ";
        cin >> bt[i];
    }

    // Sorting Burst Times and Process IDs using Selection Sort
    for (int i = 0; i < n; i++) {
        int pos = i;
        for (int j = i + 1; j < n; j++) {
            if (bt[j] < bt[pos]) pos = j;
        }
        swap(bt[i], bt[pos]);
        swap(p[i], p[pos]);
    }

    wt[0] = 0;
    for (int i = 1; i < n; i++) {
        wt[i] = wt[i - 1] + bt[i - 1];
    }

    cout << "\nProcess\tBT\tWT\tTAT\n";
    for (int i = 0; i < n; i++) {
        tat[i] = bt[i] + wt[i];
        avg_wt += wt[i];
        avg_tat += tat[i];
        cout << "P[" << p[i] << "]\t" << bt[i] << "\t" << wt[i] <<
"\t" << tat[i] << endl;
    }

    cout << "\nAverage Waiting Time: " << avg_wt / n;
    cout << "\nAverage Turnaround Time: " << avg_tat / n << endl;

    return 0;}
```

Output / Result:

```
Enter total number of processes: 4
Enter Burst Time for each process:
P[1]: 6
P[2]: 8
P[3]: 7
P[4]: 3
```

Process	BT	WT	TAT
P[4]	3	0	3
P[1]	6	3	9
P[3]	7	9	16
P[2]	8	16	24

```
Average Waiting Time: 7
Average Turnaround Time: 13
```

Conclusion:

By using C++, we successfully implemented the SJF algorithm. Sorting the processes by burst time allowed us to achieve a more efficient execution schedule than FCFS, proving that processing shorter tasks first reduces the overall wait time for the system.

Discussion:

We discussed how SJF behaves when two processes have the same burst time; in such cases, the FCFS rule is applied to break the tie. We also noted that while SJF is optimal on paper, it is difficult to implement in real-time operating systems because the CPU cannot always predict the exact burst time of a process before it begins execution.

Experiment No: 09

Experiment Name: Implementation of Banker's Algorithm for Deadlock Avoidance.

Objective:

1. To implement the Banker's algorithm to ensure resource safety.
2. To understand the calculation of the Need matrix and the discovery of a Safe Sequence.

Theory:

The Banker's Algorithm is used by the OS to avoid deadlocks. It simulates the allocation of resources to processes and checks whether the system stays in a "Safe State." A state is safe if there exists a sequence in which all processes can eventually finish their execution without a deadlock occurring.

- Available: Number of instances of each resource type currently available.
- Max: The maximum demand of each process.
- Allocation: The resources currently allocated to each process.
- Need: The remaining resource needs of each process ($\text{Need} = \text{Max} - \text{Allocation}$).

Code / Command :

```
C++

#include <iostream>
using namespace std;

int main() {
    int n, m; // n = processes, m = resource types
    cout << "Enter number of processes: "; cin >> n;
    cout << "Enter number of resource types: "; cin >> m;

    int alloc[n][m], max[n][m], avail[m], need[n][m];

    cout << "Enter Allocation Matrix:\n";
    for(int i=0; i<n; i++) for(int j=0; j<m; j++) cin >> alloc[i][j];

    cout << "Enter Max Matrix:\n";
    for(int i=0; i<n; i++) for(int j=0; j<m; j++) cin >> max[i][j];

    cout << "Enter Available Resources: ";
    for(int i=0; i<m; i++) cin >> avail[i];

    // Calculate Need Matrix
    for(int i=0; i<n; i++)
        for(int j=0; j<m; j++)
            need[i][j] = max[i][j] - alloc[i][j];
```

```

int finish[n] = {0}, safeSeq[n], count = 0;
int work[m];
for(int i=0; i<m; i++) work[i] = avail[i];

while (count < n) {
    bool found = false;
    for (int p = 0; p < n; p++) {
        if (finish[p] == 0) {
            int j;
            for (j = 0; j < m; j++)
                if (need[p][j] > work[j]) break;

            if (j == m) {
                for (int k = 0; k < m; k++) work[k] += alloc[p][k];
                safeSeq[count++] = p;
                finish[p] = 1;
                found = true;
            }
        }
    }
    if (!found) {
        cout << "System is not in a safe state!";
        return 0;
    }
}

cout << "\nSystem is in a SAFE State.\nSafe Sequence: ";
for (int i = 0; i < n; i++) cout << "P" << safeSeq[i] << (i == n-
1 ? "" : " -> ");
cout << endl;

return 0;
}

```

Output / Result:

```

Enter number of processes: 5
Enter number of resource types: 3
... (Inputs for matrices) ...
System is in a SAFE State.
Safe Sequence: P1 -> P3 -> P4 -> P0 -> P2

```

Conclusion:

The Banker's Algorithm was successfully implemented using C++. It provides a reliable way for an operating system to prevent deadlocks by checking the safety of a resource request before granting it.

Discussion:

We discussed that while the algorithm effectively prevents deadlocks, it is computationally expensive because it must run every time a process requests resources. We also touched upon the fact that in modern systems, deadlock *detection* is sometimes preferred over *avoidance* to improve performance.

Experiment No: 10

Experiment Name: Implementation of Resource Allocation Graph (RAG) for Deadlock Detection.

Objective:

1. To understand the representation of processes and resources in a graph.
2. To learn how to identify request and assignment edges.
3. To implement a logic in C++ that detects cycles in a single-instance Resource Allocation Graph.

Theory:

A Resource Allocation Graph (RAG) is a directed graph used to track which resources are held by which processes and which processes are waiting for specific resources.

1. Vertices (V): Divided into two types: Processes (represented as circles) and Resources (represented as squares).
2. Edges (E):
 - a. Request Edge ($P_i \rightarrow R_j$): Indicates that process P_i has requested resource R_j and is currently waiting for it.
 - b. Assignment Edge ($R_j \rightarrow P_i$): Indicates that resource R_j has been allocated to process P_i .

In a system where each resource type has only a single instance, the existence of a cycle in the graph is a necessary and sufficient condition for a deadlock. If the graph is a Directed Acyclic Graph (DAG), the system is deadlock-free.

Procedure / Algorithm:

1. Represent the graph using an Adjacency Matrix or Adjacency List.
2. Input the number of processes and resources.
3. Define the edges: if a process requests a resource, add an edge from Process ID to Resource ID. If a resource is held, add an edge from Resource ID to Process ID.
4. Use a Cycle Detection Algorithm (like Depth First Search - DFS) to traverse the graph.
5. If the DFS visits a node that is already in the current recursion stack, a cycle is detected.
6. Display whether a deadlock exists based on the cycle detection result.

Code / Command :

```
C++

#include <iostream>
#include <vector>

using namespace std;

bool isCyclicUtil(int v, vector<bool>& visited, vector<bool>& recStack,
vector<vector<int>>& adj) {
    if (!visited[v]) {
        visited[v] = true;
        recStack[v] = true;

        for (int i : adj[v]) {
            if (!visited[i] && isCyclicUtil(i, visited, recStack, adj))
                return true;
            else if (recStack[i])
                return true;
        }
    }
    recStack[v] = false;
    return false;
}

int main() {
    int V, E;
    cout << "Enter total number of vertices (Processes + Resources): ";
    cin >> V;
    cout << "Enter number of edges: ";
    cin >> E;

    vector<vector<int>> adj(V);
    cout << "Enter edges (Source Destination):\n";
    for (int i = 0; i < E; i++) {
        int u, v;
        cin >> u >> v;
        adj[u].push_back(v);
    }

    vector<bool> visited(V, false);
    vector<bool> recStack(V, false);
    bool deadlock = false;

    for (int i = 0; i < V; i++) {
        if (isCyclicUtil(i, visited, recStack, adj)) {
            deadlock = true;
            break;
        }
    }
}
```



```
    if (deadlock)
        cout << "\nResult: Cycle Detected! Deadlock is present in the
system.\n";
    else
        cout << "\nResult: No Cycle Detected. The system is Deadlock-
free.\n";

    return 0;
}
```

Output / Result:

```
Enter total number of vertices (Processes + Resources): 4
Enter number of edges: 4
Enter edges (Source Destination):
0 1  (P0 requests R1)
1 2  (R1 held by P1)
2 3  (P1 requests R2)
3 0  (R2 held by P0)

Result: Cycle Detected! Deadlock is present in the system.
```

Conclusion:

Through this experiment, we successfully converted a theoretical Resource Allocation Graph into a computational model. We learned that deadlock detection in single-instance systems is essentially a graph problem. By implementing a cycle-detection algorithm, we can programmatically determine system stability.

Discussion:

We discussed a critical distinction: in a multi-instance resource system, a cycle is a *necessary* but not sufficient condition for a deadlock. This means a cycle might exist without a deadlock if some resources can still be released. However, for this lab's scope (single instance), the DFS approach is highly efficient. We also noted that the complexity of this detection is $O(V + E)$, making it relatively fast for small to medium-sized systems.